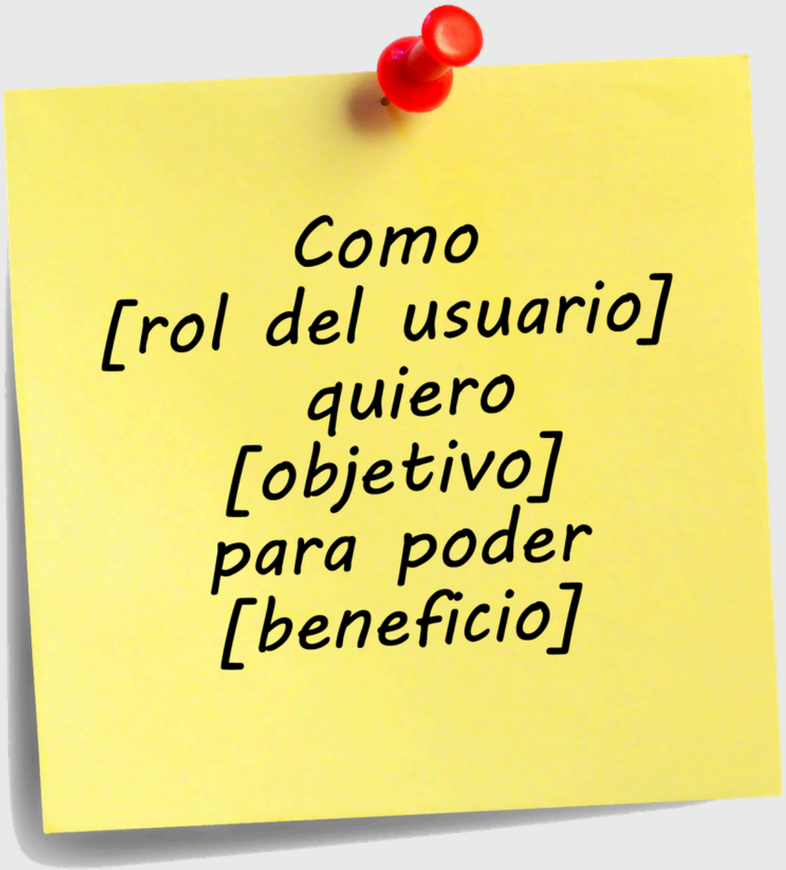
UNIVERSIDAD ORT
Uruguay

# User Stories / Use Cases

Felipe Lecot, Nahuel Merlo, Emiliano Salvo

# HISTORIAS DE USUARIO

# Historias de Usuario




## Definicion

Son una herramienta que agiliza la administración de requisitos.

## Objetivo

Describen, en una o dos frases, una funcionalidad de software desde el punto de vista del usuario, con el lenguaje que éste emplearía

## Forma

A la hora de escribir historias es preferible no adoptar formatos rigidos.
Se tienen campos que se consideran estrictamente necesarios , el “molde”, y campos arbitrarios que provean informacion util.

# Buenas Practicas

## Creacion (3 C's)

***Card**: Cada historia se reduce a un post-it (card) tal que sea facil de memorizar. La tarjeta representa una promesa a una conversacion futura*

***Conversation:** Los equipo de desarrollo tecnico y de producto definen criterios de aceptaciona cada tarjeta antes de implementar.*

***Confirmation:** El equipo de producto confirma que el equipo de desarrollo ha entendido los requisitos.*

# Historias de Usuario

## Tareas

Es el producto de las historias de usuario, describen cómo construir y no el que.

## Epicas

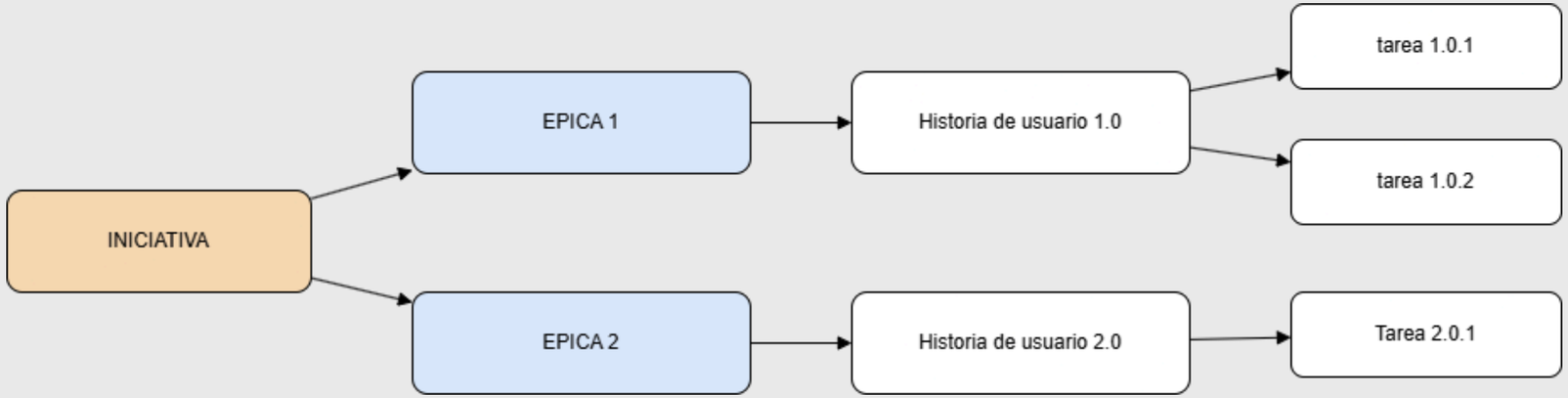
Es una historia de usuario de gran tamaño y mayor grado de incertidumbre.

## Temas

Es una coleccion de epicas e historias relacionadas que describen un sistema o subsistema.

## Iniciativa

Varias Epicas constituyen una iniciativa



# Anatomia

## Descripcion

- ¿Quién se beneficia?
- ¿Qué se quiere?
- ¿Cuál es el beneficio?

## Estimacion

Aproximación del tamaño relativo del trabajo necesario para implementar la historia. Puede estimarse en escalas relativas.

## Valor de negocio

Aporta información relacionada a la lógica de negocio. Aproximación del tamaño relativo del trabajo necesario para implementar la historia. Puede estimarse en escalas relativas.

# Anatomia

## Prioridad

Establece el orden de implementación de las historias.

**Must** have
**Should** have
**Could** have
**Won't** have

## Criterios de aceptación

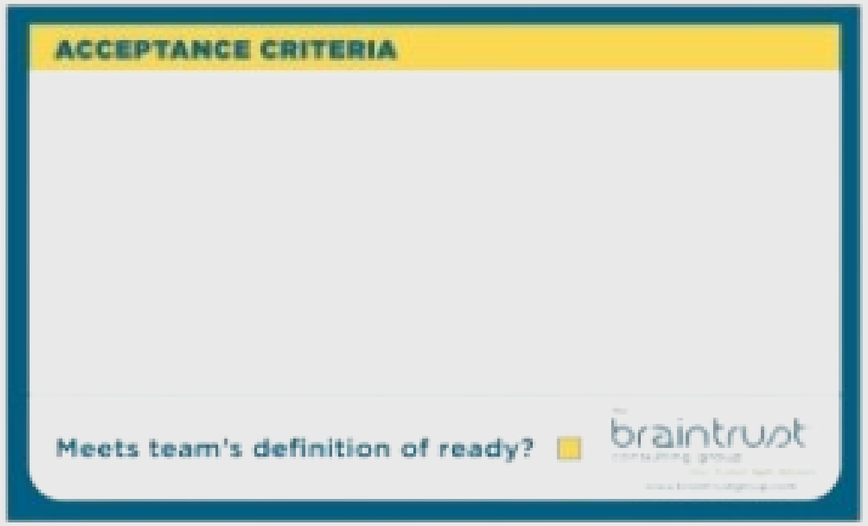
El propietario de producto comprobará si cada una de las historias de usuario se puede dar por finalizada.

# Modelo INVEST

- *Independiente*
- *Negociable*
- *Valiosa*
- *Estimable*
- *Small*
- *Testeable*

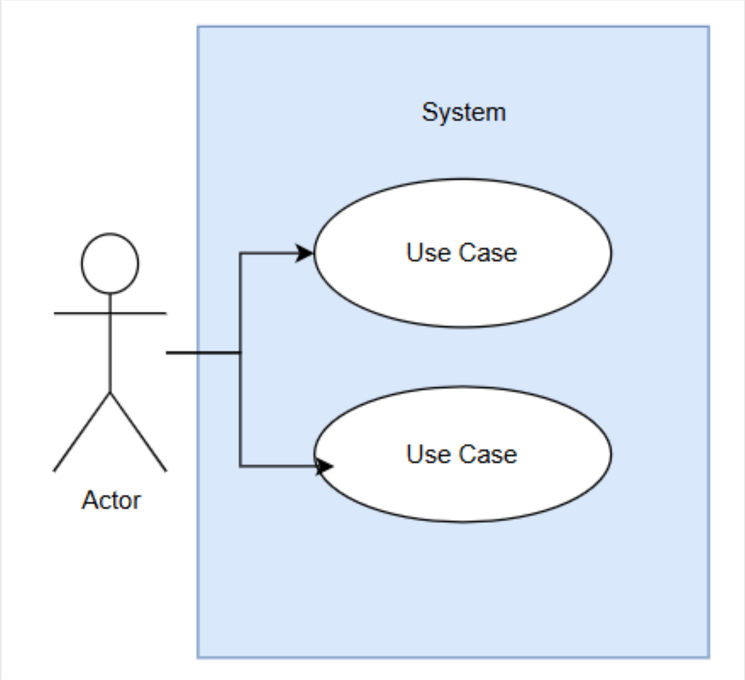
# Método SMART

- *Specific (Específicos)*
- *Measurable (Medibles)*
- *Achievable (Alcanzables)*
- *Relevant (Relevantes)*
- *Time-boxed (Limitados en el tiempo)*



# CASOS DE USO

# Casos de uso




## Definicion

Son otra técnica más que a veces se incorpora en los procesos ágiles, pero no son equivalentes a las historias de usuario.

## Objetivo

Capturan la funcionalidad deseada desde la perspectiva de los usuarios (actores) y cómo interactúan con el sistema.

## Forma

Se representan usando UML (unified modeling language) en los diagramas de casos de uso.

# Componentes

# Casos de uso

- Actores: los roles que interactúan con el sistema.
- Precondiciones: estado del sistema antes de ejecutar el caso de uso.
- Flujo principal: secuencia de pasos en el escenario exitoso.
- Flujos alternativos: variaciones del flujo principal.
- Flujos de excepción: qué ocurre cuando algo falla.
- Postcondiciones: estado del sistema después de ejecutar el caso de uso.

# Plantilla para casos de uso

| [Identificador] | [nombre descriptivo] | | |
|---|---|---|---|
| **Descripción** | [Descripción corta de quién (actor) va a hacer, por qué, y qué obtendrá | | |
| **Actores** | [Actores involucrados en el caso de uso] | | |
| **Pre condiciones** | [Condiciones que se cumplen antes de iniciar el caso de uso] | | |
| **Post condiciones** | [Condiciones a cubrir después de terminado el caso de uso] | | |
| **Secuencia Normal** | **#** | **Acción (actor)** | **Reacción (sistema)** |
| | 1 | [lo que realiza el actor] | [lo que responde el sistema] |
| | 2 | [siguiente paso que hace actor] | [siguiente respuesta del sistema] |
| | | • 2.1 Si [Situación que produce una alternativa] el sistema deberá {<acción a realizar>, realizar el caso de uso [caso de uso]}<br>• .... | • 2.2 Si [Situación que produce una alternativa] el sistema deberá {<acción a realizar>, realizar el caso de uso [caso de uso]}<br>• .... |
| | ... | | ... |
| | n | | ... |
| **Excepciones** | **#** | **Acción (actor)** | **Reacción (sistema)** |
| | p | En el caso de que [situación que provoca la excepción] el sistema deberá {<acción a realizar>, realizar el caso de uso [caso de uso]} | En el caso de que [situación que provoca la excepción] el sistema deberá {<acción a realizar>, realizar el caso de uso [caso de uso]} |
| | ... | | ... |
| | q | | ... |
| **Rendimiento** | El sistema deberá realizar la/s acción /es descrita/s en {los pasos [primer paso] al [último paso], el paso [número de paso]} en un máximo de [cota de tiempo] | | |
| **Frecuencia** | Este caso de uso se espera que se lleve a cabo una media de [número de veces] al [unidad temporal] | | |
| **Importancia** | {vital, importante quedaría bien} | | |
| **Urgencia** | {inmediatamente, hay presión, puede esperar} | | |
| **Comentarios** | <otras consideraciones en formato libre> | | |

# Casos de uso

| Aspecto | Casos de uso | Historias de usuario |
|---|---|---|
| **Lenguaje** | UML (técnico, formal) | Lenguaje natural del usuario |
| **Enfoque** | Especificación de requisitos (el cómo) | Captura de requisitos (el qué) |
| **Nivel de detalle** | Alto (flujos, extensiones, precondiciones) | Bajo (placeholder para conversación) |
| **Documentación** | Diagramas UML + descripciones extensas | Tarjeta + criterios de aceptación simples |
| **Momento de elaboración** | Análisis previo extenso | *Just-in-time*, poco antes del desarrollo |
| **Criterios de aceptación** | Matrices de seguimiento con porcentajes | Binario (cumple o no cumple) |
| **Mantenimiento** | Requiere actualización de diagramas UML | Mínimo (se descartan tras implementar) |
| **Autor principal** | Analista funcional o arquitecto | Product owner + equipo + usuario |
| **Flexibilidad ante cambios** | Requiere reelaboración formal | Alta, cambios incorporados fácilmente |

## ¿Qué es UML?

Es un lenguaje de modelado para describir, de forma sencilla, un sistema desde sus perspectivas estática y dinámica.

## ¿Cómo funciona?

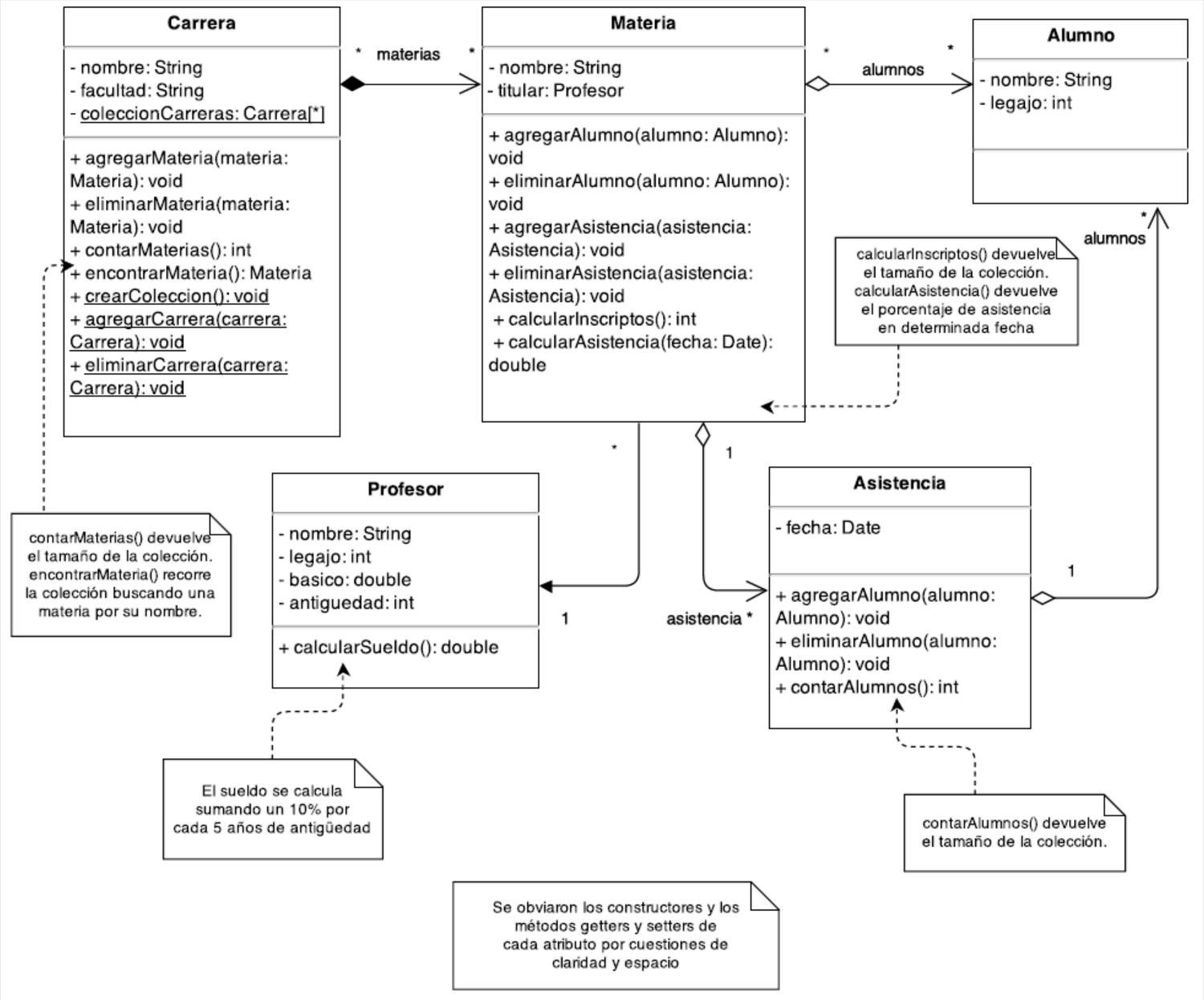
Utiliza principalmente notaciones gráficas para expresar el diseño de los proyectos de software

## ¿Para qué?

Su uso facilita la comunicación entre los equipos de proyecto, la exploración de posibles diseños y la validación de la arquitectura del software.

# Ejemplo

Diagrama de clases



# Demostracion practica

**Como** Bibliotecario,
**quiero** registrar el préstamo de un libro a un socio,
**para** mantener un control de quién tiene cada ejemplar y cuándo debe devolverlo.

# Caso de Uso 1

| #1 | Registrar Préstamo |
|---|---|
| **Descripción** | El bibliotecario ingresa el ID del socio y el ID del libro para crear el registro. |
| **Actores** | Bibliotecario, socio |
| **Precondiciones** | El socio existe en el sistema y no debe nada<br>El libro existe y esta disponible |
| **Post condiciones** | Se crea un nuevo préstamo<br>Se registra: fechaInicio y  fechaFin=fechaInicio + 7 días<br>El libro pasa a no disponible |

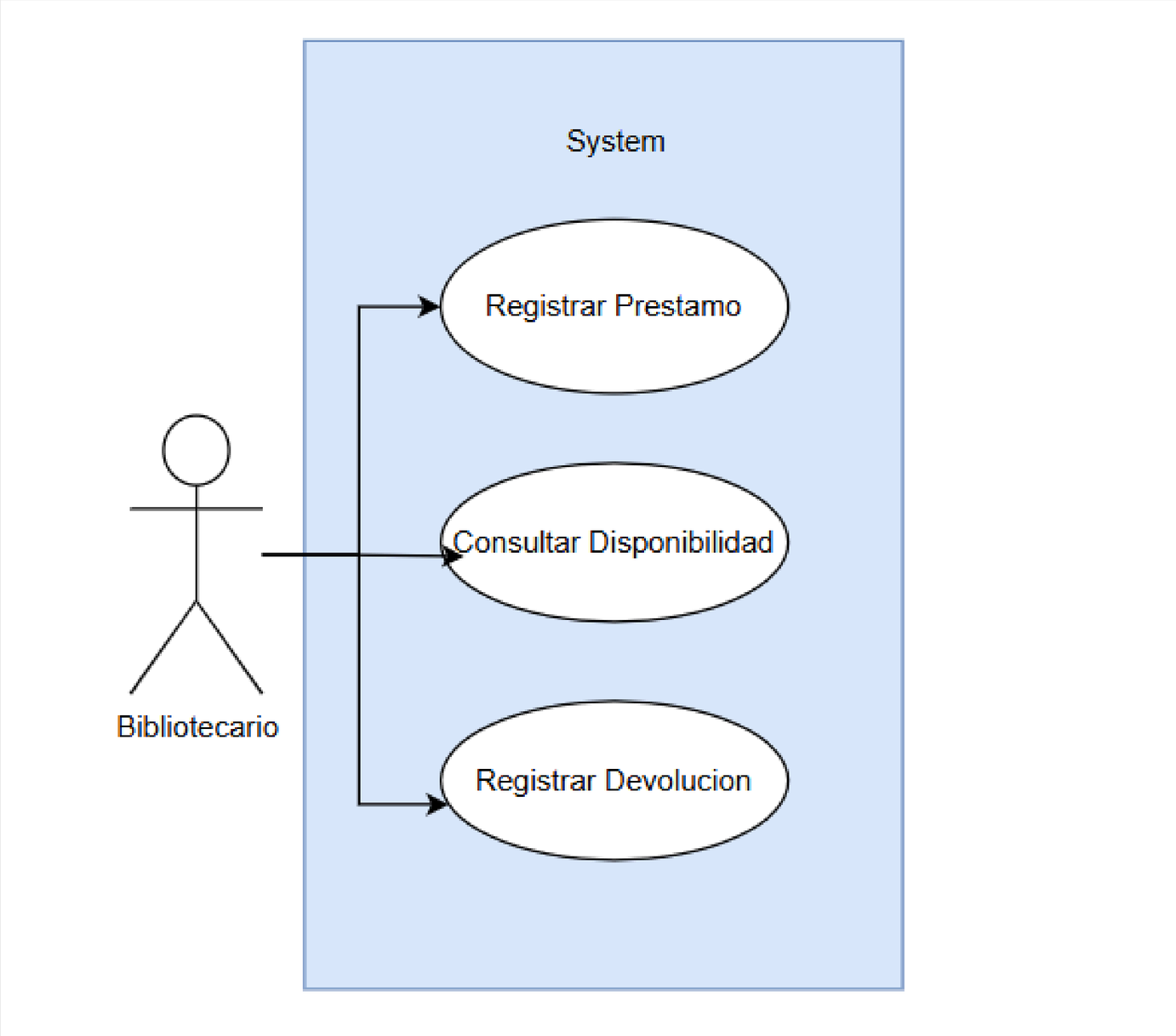
# Caso de Uso 2

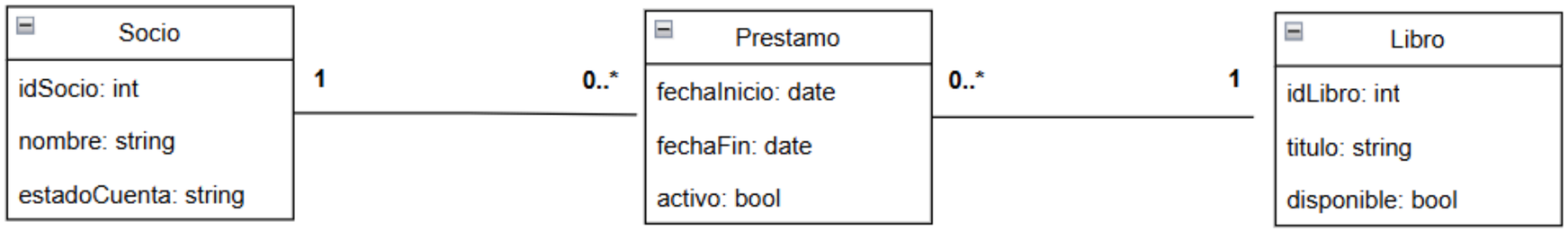
| #2 | Consultar Disponibilidad |
|---|---|
| **Descripción** | El sistema verifica si el libro está en estante o ya está prestado. |
| **Actores** | Bibliotecario |
| **Precondiciones** | El libro existe |
| **Post condiciones** | Se informa si el libro está disponible o no |

# Caso de Uso 3

| #3 | Registrar Devolución |
|---|---|
| **Descripción** | El bibliotecario marca el libro como disponible nuevamente al recibirlo. |
| **Actores** | Bibliotecario |
| **Precondiciones** | Existe un préstamo activo para ese libro |
| **Post condiciones** | El préstamo se marca como finalizado<br>El libro vuelve a estar disponible |

# Casos de Uso


System
Registrar Prestamo
Consultar Disponibilidad
Registrar Devolucion
Bibliotecario


Socio
idSocio: int
nombre: string
estadoCuenta: string
1
0..*
Prestamo
fechaInicio: date
fechaFin: date
activo: bool
0..*
1
Libro
idLibro: int
titulo: string
disponible: bool

# Historias de usuario en GitHub Issues + Projects